

A. Two Sum Trouble

time limit per test: 1 second memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

Your little brother, Bob, loves playing with integers. One day, his teacher gave him a sorted list of N integers in non-decreasing order. Now, your brother wants to play a game with you.

Bob will give you an integer S . You have to find if it is possible to find two values from the list (at distinct positions) whose sum is equal to S .

Since you are feeling very tired, you decide to write a program that can quickly answer Bob's query.

Input

The first line contains two integers N ($1 \leq N \leq 10^6$) and S ($1 \leq S \leq 10^9$), denoting the length of the list, and the target Sum.

In the next line, there will be N integers $a_1, a_2, a_3 \dots a_n$ ($1 \leq a_i \leq 10^9$) in non-decreasing order, separated by spaces.

Output

Print two distinct 1-based indices i and j such that $a_i + a_j = S$ where $i < j$. If no such pair exists, then print -1. If multiple solutions exist, you may print any one of the valid answers.

Examples

Input

```
4 10
1 3 5 7
```

Output

```
2 4
```

Input

```
6 18
1 5 8 9 9 10
```

Output

```
3 6
```

Input

```
4 7
2 4 6 8
```

Output

```
-1
```

Input

```
4 10
1 5 6 8
```

Output

```
-1
```

Note

In the second sample input, 4 5 is also a valid output.

B. Two Sum Revisited

time limit per test: 1 second memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given two integer arrays A and B of sizes N and M , respectively, and an integer K . Both arrays are sorted in non-decreasing order. Your task is to find any pair of indices (i, j) such that:

- i is a valid index in array A ($1 \leq i \leq N$)
- j is a valid index in array B ($1 \leq j \leq M$)
- the sum $A[i] + B[j]$ is closest to K (i.e., it minimizes $|A[i] + B[j] - K|$).

Input

The first line contains Three integers N , M and K ($1 \leq N, M \leq 2 \times 10^5, -10^9 \leq K \leq 10^9$).

The second line contains N integers — the elements of array A ($-10^9 \leq A_i \leq 10^9$).

The third line contains M integers — the elements of array B ($-10^9 \leq B_j \leq 10^9$).

Output

Print two space-separated integers i and j the 1-based indices of the chosen pair.

If there are multiple answers, output any.

Examples

Input

```
4 4 0
-5 -2 -1 5
-5 0 1 1
```

Output

```
3 4
```

Input

```
6 6 1
-5 -3 3 4 4 5
-2 0 2 2 3 5
```

Output

```
3 1
```

Input

```
1 1 8
-2
-8
```

Output

```
1 1
```

Input

```
2 2 -4
-7 4
-5 4
```

Output

```
1 2
```

Input

```
3 4 -2
-15 -15 8
-14 4 4 6
```

Output

```
3 1
```

C. A Beautiful Sorted List

time limit per test: 1 second memory limit per test: 1024 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

Alice and Bob are two friends. Alice has a list of length N in non-decreasing order, and Bob has a list of length M , also in non-decreasing order.

Now, they want to combine their lists into a single non-decreasing list of length $N+M$. However, they are not very good at algorithms, so they asked for your help.

Since you are a computer science student, your task is to write an efficient algorithm to merge the two given lists into one non-decreasing list. Solve the problem in $O(N+M)$.

Input

The first line contains an integer N ($1 \leq N \leq 10^6$), denoting the length of Alice's list.

The second line contains N space-separated integers representing Alice's list.

The third line contains an integer M ($1 \leq M \leq 10^6$), denoting the length of Bob's list.

The fourth line contains M space-separated integers representing Bob's list.

All the numbers given in the input will fit within a 32-bit signed integer. It is guaranteed that the given lists will be in non-decreasing order.

Output

You have to make a sorted list in non-decreasing order from the given lists and show the output.

Examples

Input

```
4
1 3 5 7
4
2 2 4 8
```

Output

```
1 2 2 3 4 5 7 8
```

Input

```
3
2 10 12
6
3 4 6 7 8 9
```

Output

```
2 3 4 6 7 8 9 10 12
```

Input

```
5
1 2 3 4 5
2
10 12
```

Output

```
1 2 3 4 5 10 12
```

Input

```
4
1 2 12 13
3
10 15 18
```

Output

```
1 2 10 12 13 15 18
```

Input

```
8
1 2 3 8 8 10 12 14
9
1 1 4 5 6 8 13 15 16
```

Output

```
1 1 1 2 3 4 5 6 8 8 8 10 12 13 14 15 16
```

D. Longest Subarray Sum

time limit per test: 1 second memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given an array of N integers and an integer K . Your task is to find the length of the longest contiguous subarray whose sum is less than or equal to K .

Input

The first line contains two integers N ($1 \leq N \leq 10^5$) and K ($1 \leq K \leq 10^9$) — the size of the array and the maximum allowed sum.

The second line contains N space-separated integers $a_1, a_2, a_3 \dots a_n$ ($1 \leq a_i \leq 10^6$) — the elements of the array.

Output

Print a single integer — the length of the longest contiguous subarray whose sum is less than or equal to K .

Examples

Input

```
5 4
4 1 2 1 5
```

Output

```
3
```

Input

```
5 5
1 1 1 1 1
```

Output

```
5
```

Input

```
3 1
2 3 4
```

Output

```
0
```

Input

```
10 12
1 2 6 4 3 2 3 1 4 2
```

Output

```
5
```

Note

In the first example, possible subarrays with sum less than or equal to 4 are [4], [1], [2], [1], [1, 2], [2, 1], [1, 2, 1]. Among them, the longest size is 3.

In the second example, sum of the entire array is 5. Hence, we can take the whole array.

In the third example, no subarray has sum less than or equal to 1. Hence, the answer is 0.

E. Longest K-Distinct Subarray

time limit per test: 1 second memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given an array of integers of length N and an integer K . Your task is to find the length of the longest contiguous subarray that contains at most K distinct elements.

Input

The input consists of:

The first line contains two integers N and K — the size of the array and the maximum number of distinct elements allowed ($1 \leq N \leq 2 \times 10^5, 1 \leq K \leq N$).

The second line contains N space-separated integers $A_1, A_2, A_2 \dots A_n$ — the elements of the array ($1 \leq A_i \leq N$).

Output

Print a single integer — the length of the longest contiguous subarray that contains at most K distinct elements.

Examples

Input

```
4 1
2 1 2 4
```

Output

```
1
```

Input

```
6 2
6 6 5 6 1 2
```

Output

```
4
```

Input

```
1 1
1
```

Output

```
1
```

Input

```
2 2
1 2
```

Output

```
2
```

F. Count the Numbers

time limit per test: 1 second memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given a sorted array a of n elements, and some queries. In each query, you are given a pair $[x, y]$ and you have to count how many numbers a_i are there such that $x \leq a_i \leq y$. For example, if the array is $[10, 20, 20, 45, 79]$ and you are given a query $[20, 50]$, then answer will be 3 because there are in total 3 numbers that's value is between 20 and 50.

Input

The first line of the input contains $n(1 \leq n \leq 10^5)$ and $q(1 \leq q \leq 10^5)$ denoting the array size and the number of queries respectively. The next line will contain the array elements separated by space where $1 \leq a_i \leq 10^9$ where $i = 0, 1, 2, \dots, n - 1$. Each of the next q lines will contain a pair $[x, y]$ where $1 \leq x \leq y \leq 10^9$. See the sample input format for better understanding.

Note1: It is guaranteed that the given array is sorted in non-decreasing order.

Note2: It is also guaranteed that the queries are valid. Which means, for each query $[x, y]$, $x \leq y$.

Output

For each query $[x, y]$, output a single integer P denoting the number of elements in the array a such that $x \leq a_i \leq y$.

Example

Input

```
5 3
10 20 20 45 79
20 50
5 45
1 100
```

Output

```
3
4
5
```

G. Can You Split the Array?

time limit per test: 1 second memory limit per test: 256 megabytes

You are given an array containing N positive integers. Your task is to partition the array into at most K subarrays so that the maximum sum among all subarrays is as small as possible.

Input

The first line contains two integers N ($1 \leq N \leq 2 \times 10^5$) and K ($1 \leq K \leq N$), the size of the array and the number of subarrays in the partition.

The next line contains N integers $a_1, a_2, \dots, a_N$ ($1 \leq a_i \leq 10^6$), the elements of the array.

Output

Print one integer: the minimum possible largest subarray sum.

Examples

Input

```
5 3
1 2 3 4 5
```

Output

```
6
```

Input

```
5 3
1 5 1 1 5
```

Output

```
6
```

Note

Brief Explanation for First Sample: There are many ways to partition the array

- The optimal/best partition is $[1, 2, 3], [4], [5]$. The maximum sum for this partition is 6. This is because
 - the sum of the first subarray $[1, 2, 3]$ is $1 + 2 + 3 = 6$;
 - the sum of the second subarray $[4]$ is 4; and
 - the sum of the third subarray $[5]$ is 5.
- Another partition is $[1, 2], [3, 4], [5]$. The maximum sum for this partition is 7. This is because
 - the sum of the first subarray $[1, 2]$ is $1 + 2 = 3$;
 - the sum of the second subarray $[3, 4]$ is $3 + 4 = 7$;
 - the sum of the third subarray $[5]$ is 5.

Since we want to find the smallest possible value for the maximum sum, the first partitioning ($[1, 2, 3], [4], [5]$) is the best. The answer is 6.

Problem Extension: What if we allow negative numbers? How would the solution change?

H. Can You Cut the Ropes?

time limit per test: 1 second memory limit per test: 256 megabytes

You are given n ropes with positive integer lengths. You may cut the ropes into smaller pieces (each with positive integer length).

Your task is to obtain at least k pieces such that every piece has the same positive integer length. Any leftover part after cutting may be discarded.

Find the maximum possible length of each piece. If it is not possible to obtain at least k pieces of any positive integer length, print -1 .

Input

The first line contains two integers n ($1 \leq n \leq 2 \times 10^5$) and k ($1 \leq k \leq 10^{18}$), the number of ropes and the required number of pieces respectively.

The next line contains n integers $l_1, l_2, \dots, l_n$ ($1 \leq l_1, \dots, l_n \leq 10^9$), the lengths of the ropes.

Output

Print one integer: the maximum possible length of each piece.

If it is not possible, print -1 .

Examples

Input

```
4 8
2 6 4 4
```

Output

```
2
```

Input

```
5 1000
1 2 3 4 5
```

Output

```
-1
```

Note

Brief Explanation for the First Sample: All of our final ropes have to be of the same length.

- Suppose that we want our final ropes to have length 1. We can get $2 + 6 + 4 + 4 = 12$ pieces of ropes of length 1. We can
 - cut the first rope into $\frac{2}{1} = 2$ pieces of length 1,
 - cut the second rope into $\frac{6}{1} = 6$ pieces of length 1,
 - cut the third rope into $\frac{4}{1} = 4$ pieces of length 1, and
 - cut the fourth rope into $\frac{4}{1} = 4$ pieces of length 1.
- Suppose that we want to our final ropes to have length 2. We can get $1 + 3 + 2 + 2 = 8$ pieces of ropes of length 2. We can
 - cut the first rope into $\frac{2}{2} = 1$ piece of length 2,

- cut the second rope into $\frac{6}{2} = 3$ pieces of length 2,
 - cut the third rope into $\frac{4}{2} = 2$ pieces of length 2, and
 - cut the fourth rope into $\frac{4}{2} = 2$ pieces of length 2.
- Suppose that we want our final ropes to have length 3. We can get $0 + 2 + 1 + 1 = 4$ pieces of ropes of length 3.
 - $\frac{2}{3} = 0.667$, so we can get 0 pieces of length 3;
 - $\frac{6}{3} = 2$, so we can get 2 pieces of length 3;
 - $\frac{4}{3} = 1.33$, so we can get 1 piece of length 3; and
 - $\frac{4}{3} = 1.33$, so we can get 1 piece of length 3.

To summarize, we can get 12 pieces of ropes of length 1, 8 pieces of ropes of length 2, and 4 pieces of ropes of length 3. Since we need at least $k = 8$ pieces, the answer can not be 3. It also can not be 1, as we want the maximum length. So the answer is 2.